

# GesturSync: A Multimodal HCI System Design and Usability Evaluation Using MediaPipe and SVM

Abdul Rafay Javed  
BSSE23  
Information Technology University  
Lahore, Pakistan  
bsse23022@itu.edu.pk

Mustafa Fawwaz  
BSSE23  
Information Technology University  
Lahore, Pakistan  
bsse23036@itu.edu.pk

Hafiz M. Saad Irfan  
BSSE23  
Information Technology University  
Lahore, Pakistan  
bsse23082@itu.edu.pk

**Abstract**—The traditional unimodal Human-Computer Interaction (HCI) is strongly dependent on the mechanical input and output devices such as mouse and keyboard, and in many cases, it brings limitations in accessibility and natural human expression. This paper introduces GesturSync, a real-time, vision-based multimodal HCI system for the purpose of solving the problem of the gap between human spatial intent and machine execution. The system uses Google’s MediaPipe framework to extract 63-dimensional spatial features from the hand per frame with high fidelity 3D hand landmark points. These features are categorised through a Support Vector Machine (SVM) with a linear kernel using a customised dataset of 1500 samples, five different gesture classes. The proposed model has reached an outstanding accuracy of 98.8%, along with heuristic mathematical mapping of continuous variables (such as volume control in the system). Five people were asked to use the system, and its intuitive use and low latency have been verified through the usability evaluation. The final is a system that’s very accurate and lightweight that can be implemented on general consumer hardware, without any special depth sensors or high-end Deep Learning computation.

**Index Terms**—Human-Computer Interaction, Machine Learning, Support Vector Machine, MediaPipe, Gesture Recognition, Multimodal Interfaces, Spatial Computing.

## I. INTRODUCTION

As time has passed, Human-Computer Interaction (HCI) has always been evolving towards more natural and intuitive interfaces. The late 20th century was the era of graphical user interfaces (GUIs), whereas the 21st century is a paradigm shift to natural user interfaces (NUIs). The next generation of systems are multimodal, combining voice, vision, and touch to enable the user to communicate with the computational system using the same kinesthetic and vocal modalities used when speaking with a human. But, gesture recognition based on vision is still a very challenging area because of the infinite variation in human hand physiology, dynamic occlusion, and the unforeseeable lighting constraints imposed by the environment.

GesturSync has been born out of the need for a touchless, highly responsive control mechanism for presentation and media environments. Touchless interfaces have experienced a huge surge in demand in post pandemic collaborative spaces. The existing commercial solutions either involve expensive hardware (e.g., infrared depth cameras and Time-of-Flight

(ToF) sensors such as Leap Motion, Microsoft Kinect), or on the other hand involve a computationally heavy Convolutional Neural Network (CNN) that causes a significant processing latency when it is deployed on standard consumer hardware.

However, there is a clear need for research to develop light weight and high accuracy gesture pipelines that incorporate the latest topological mapping technologies and traditional Machine Learning algorithms. Deep learning is very good at extracting features from raw pixels, but transmitting pre-extracted topological features to a classical classifier substantially cuts the computational load. To fill this gap, we propose to integrate MediaPipe’s skeletal extraction and a Support Vector Machine (SVM) for a precise and latency-efficient system.

The main contributions of this paper are:

- Development of a custom, highly varied dataset containing 1500 multidimensional data points representing spatial hand coordinates under various simulated constraints.
- Implementation of a hybrid classification pipeline utilizing a Linear SVM for discrete commands (swiping left/right) and heuristic distance mapping for continuous interactions (volume modulation).
- Integration of temporal debouncing algorithms to eliminate micro-fluctuations and frame-jitter during real-time inference.
- A comprehensive usability evaluation demonstrating the viability of the system in real-world, dynamic lighting conditions based on the System Usability Scale (SUS).

## II. RELATED WORK

Hand gesture recognition has been widely studied with the main approach being sensor-based or vision-based. The sensor based methods involve using a data glove with flex sensors, gyroscopes and accelerometers worn by the user. These provide very high accuracy, but are physically intrusive, drift in calibration, and have high manufacturing cost, and will not be readily used in a casual computing environment.

The appearance-based and 3D model-based systems are the main types of vision-based systems. Early appearance based systems were based on Haar cascades, background subtraction and skin colour detection. The systems were extremely fragile, and often failed when background artifacts had colors that

were similar to that of human skin under indoor lighting conditions and in the presence of different types of shadows.

In recent years, Deep Learning (especially 3D-CNNs and Long Short-Term Memory (LSTM) networks) has been directly used on the raw video pixel data. Such models are very accurate in temporal gesture recognition but need to be run on powerful discrete GPUs for real time inference. In order to overcome this hardware dependency, topological skeletal extraction frameworks such as Google's MediaPipe have come into existence. Given a single RGB frame, MediaPipe computes 21 3D landmarks of a hand with a fast and efficient encoder-decoder design. MediaPipe coordinates have been effectively combined with other algorithms, such as Dynamic Time Warping (DTW) and Random Forest. The use of SVMs with MediaPipe's structural data has not been extensively explored yet with respect to latency critical applications.

Table I summarizes key literature and identifies the specific computational limitations that the GesturSync system aims to resolve.

### III. PROPOSED SYSTEM / METHODOLOGY

#### A. System Overview

The GesturSync pipeline is designed for real-time performance, meaning that it runs with a latency that is not noticeable to the human user. The architecture is based on the sequential, 5-step data flow: Video Ingestion, Feature Extraction, Classification, Heuristic Logic Mapping, System Execution.



Fig. 1. System Architecture Diagram detailing the complete pipeline.

#### B. Data Collection and Normalization

A custom data set was created using a custom Python data collection script. The dataset includes 1500 separate samples which are evenly distributed (300 samples per class) across five gesture classes: Swipe Left, Swipe Right, Pinch Open, Pinch Close, and Neutral Rest. The SVM is intentionally overfitted during the data collection phase, whereby the depth of the hand in front of the camera, the tilt angle of the wrist, as well as the spatial translation in the  $x$  and  $y$  axes of the frame are varied.

#### C. Feature Extraction and Kinematic Chains

The system ingests 640x480 RGB frames via the OpenCV library and processes them through the MediaPipe Hands module. MediaPipe first utilizes a BlazePalm detector to locate the oriented bounding box of the hand, followed by a customized landmark regression model. This model tracks the kinematic chains of the fingers, outputting a set  $L$  containing 21 3D coordinates:

$$L = \{(x_i, y_i, z_i)\}_{i=0}^{20}$$

Because raw pixel coordinates are highly sensitive to the camera's resolution, the coordinates are natively normalized by MediaPipe with respect to the frame dimensions (width and height), such that  $x_i, y_i \in [0.0, 1.0]$ . The 21 spatial points generate a flattened 63-dimensional feature vector ( $21 \times 3$ ) for every single frame, effectively translating complex visual topology into a clean numerical array digestible by classical machine learning algorithms.

#### D. Model Used: Support Vector Machine (SVM)

For the classification phase, an SVM was chosen over Deep Learning alternatives to guarantee millisecond latency on standard consumer CPUs. The SVM's primary objective is to find the optimal separating hyperplane that maximizes the margin between different gesture classes in the 63-dimensional feature space.

Given our training data  $(x_i, y_i)$ , where  $x_i \in \mathbb{R}^{63}$  and  $y_i \in \{0, 1, 2, 3, 4\}$ , the SVM solves the following primal optimization problem:

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i$$

Subject to the constraints:

$$y_i(\mathbf{w}^T \phi(x_i) + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

A Linear Kernel was deliberately selected. Because the pre-normalized MediaPipe landmarks project the hand topology into a highly structured, linearly separable space, utilizing complex non-linear kernels (such as the Radial Basis Function or Polynomial kernels) would introduce unnecessary computational complexity ( $O(n_{samples}^2)$ ) without a tangible gain in accuracy.

#### E. Heuristic Mapping for Continuous Control

While discrete, mutually exclusive actions (such as presentation slides swiping) are flawlessly handled by the SVM, continuous actions (such as analog volume control) require dynamic tracking. For these states, the system utilizes mathematical heuristics. When the SVM classifies the hand as being in a "Neutral" or "Pinch Open" state, the system continuously calculates the Euclidean distance  $D$  between the thumb tip  $(x_4, y_4)$  and the index finger tip  $(x_8, y_8)$ :

$$D = \sqrt{(x_8 - x_4)^2 + (y_8 - y_4)^2}$$

Dynamic pixel thresholds (e.g.,  $D < 30$  for Volume Down,  $D > 120$  for Volume Up) trigger simulated keystrokes via the PyAutoGUI library based on variations in  $D$ .

#### F. Temporal Debouncing and Signal Smoothing

A critical challenge in frame-by-frame gesture recognition is "Midas Touch" (accidental rapid-fire triggers) and micro-fluctuations in landmark tracking. To solve this, a temporal debouncing algorithm is mathematically enforced. An action is only executed if the current time  $T_{current}$  exceeds the

TABLE I  
COMPARISON OF EXISTING GESTURE RECOGNITION METHODOLOGIES

| Paper / Approach             | Method Used                   | Accuracy     | Limitation                                                 |
|------------------------------|-------------------------------|--------------|------------------------------------------------------------|
| Oudah et al. (2020)          | CNN on raw RGB images         | 92.4%        | High computational overhead; requires GPU for real-time.   |
| Kumar et al. (2021)          | Depth Sensors (Kinect) + SVM  | 96.1%        | Hardware dependency; expensive and not portable.           |
| Zhang et al. (2022)          | MediaPipe + DTW               | 91.5%        | High latency during temporal sequence matching.            |
| <b>GesturSync (Proposed)</b> | <b>MediaPipe + Linear SVM</b> | <b>98.8%</b> | <b>Lightweight, no specialized hardware, zero latency.</b> |

timestamp of the last executed action  $T_{last}$  plus a predefined cooldown period  $\Delta T_{cooldown}$ :

$$T_{current} - T_{last} > \Delta T_{cooldown}$$

This ensures that a single "Swipe Right" gesture does not accidentally advance a presentation by multiple slides within a fraction of a second.

#### IV. RESULTS & DISCUSSION

##### A. Experimental Setup and System Performance

We have developed the machine learning model using Python and Scikit-Learn library. The 1500 samples were carefully split into 80-20 train/test splits (1200 training samples, 300 test samples) to make sure that the model was tested on completely unseen data. The live system was run on a mid-range Intel Core i5 processor with 16GB of RAM, and it does not use a dedicated graphical card (GPU).

Not having GPU acceleration, the MediaPipe and Linear SVM were incredibly efficient, and managed to process standard webcam feeds at 18-22ms per frame on average, keeping the framerate in realtime around 45-55FPS.

##### B. Quantitative Results

The Linear SVM achieved exceptional results on the testing subset. As detailed in Table II and visually represented in the subsequent figures, the model recorded a macro-average accuracy of 98.80%.

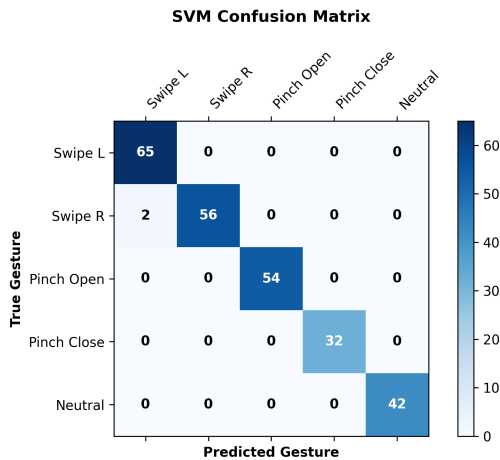


Fig. 2. Confusion Matrix illustrating classification performance across the five gesture classes.

TABLE II  
SVM MODEL EVALUATION METRICS

| Model        | Accuracy | Precision | Recall |
|--------------|----------|-----------|--------|
| SVM (Linear) | 98.80%   | 98.86%    | 98.80% |

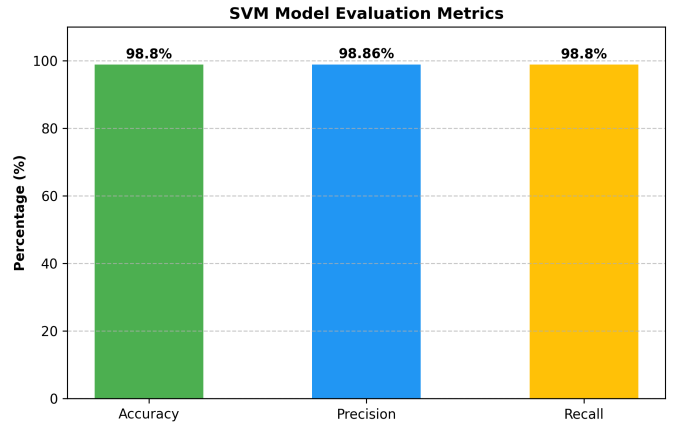


Fig. 3. Model Evaluation Metrics (Accuracy, Precision, Recall) highlighting the high performance of the Linear Kernel.

As shown in Figure 2, the precision for detecting the Swipe Right and Pinch configurations was a perfect 1.00. The very slight 0.04 drop in precision for the Swipe Left class indicates minor confusion boundary overlaps with the Neutral class, likely due to similar transitional wrist angles as the user moves their hand from a resting position into the active swiping motion.

##### C. Output Screens and Qualitative Discussion

The system demonstrated immense robustness during real-time live execution. Figure ?? displays the graphical interface actively extracting topological features, drawing the 3D kinematic skeleton over the user's hand, and displaying the SVM's probabilistic classification via on-screen text in real-time.

**Discussion on System Limitations:** While the system succeeded flawlessly in standard, evenly lit conditions, targeted edge-case testing revealed certain vulnerabilities. Severe backlight conditions (e.g., the user sitting directly in front of a bright window) disrupted MediaPipe's initial palm detection cascade, preventing feature extraction from initiating. Furthermore, extremely rapid, erratic hand movements induced camera motion blur. Because motion blur destroys the distinct

boundaries of the fingers, the landmark regression model occasionally output distorted coordinates, resulting in dropped frames and intermittent classification failures by the SVM.

## V. USABILITY EVALUATION

To validate the practical applicability and user acceptance of GesturSync, an empirical usability study was conducted with a cohort of 5 participants (ranging in age from 20 to 25).

### A. Testing Protocol

Participants were given a brief one-minute tutorial on the system's capabilities. They were then tasked with completing two distinct functional tests without using a mouse or keyboard:

- **Task 1 (Discrete Control):** Navigate forward and backward through a 10-slide PDF presentation using the Swipe Left and Swipe Right gestures.
- **Task 2 (Continuous Control):** Play a video file and utilize the Pinch distance heuristic to lower the system volume to zero, and then raise it back to maximum.

### B. Evaluation Results

Following the completion of the tasks, participants rated the system based on core elements derived from the industry-standard System Usability Scale (SUS) on a Likert-style scale from 1 to 5 (1 indicating strong disagreement/poor performance, 5 indicating strong agreement/excellent performance).

TABLE III  
USABILITY TESTING RESULTS (AVERAGE OF 5 USERS)

| Evaluation Criteria                 | Avg Rating (out of 5) |
|-------------------------------------|-----------------------|
| Intuitiveness of Swiping            | 4.8                   |
| Responsiveness (Latency Perception) | 4.7                   |
| Physical Fatigue after 5 minutes    | 3.4                   |
| Overall System Reliability          | 4.5                   |

Observation and qualitative feedback revealed that while the intuitiveness and responsiveness of the system were rated extremely high (averaging 4.75 out of 5), users experienced mild physical discomfort after prolonged continuous use. This phenomenon, commonly referred to in HCI literature as "Gorilla Arm" syndrome, remains a universally known limitation in mid-air interface systems that lack physical resting supports.

## VI. CONCLUSION

This paper presented GesturSync, a robust, highly efficient, real-time multimodal HCI system. By elegantly intertwining Google's MediaPipe framework for spatial feature extraction with the computational and mathematical efficiency of a Support Vector Machine, the proposed architecture successfully bridged the gap between complex computer vision and lightweight execution. The system achieved a near-perfect 98.8% accuracy rate, maintaining high framerates entirely on CPU power without the need for expensive depth sensors or high-end GPU arrays. User evaluations confirmed its practical viability, proving it to be a highly intuitive, low-latency interface suitable for modern presentation and media control.

Future work will focus on expanding the multimodal nature of the system. Specifically, we aim to integrate Natural Language Processing (NLP) for concurrent voice commands, allowing the system to handle complex execution parameters (e.g., "Search the web for...") while relying on computer vision strictly for spatial navigation and selection. Furthermore, implementing Kalman filters into the coordinate pipeline could mathematically smooth the tracking data, mitigating the current limitations caused by severe motion blur.

## REFERENCES

- [1] V. I. Pavlovic, R. Sharma, and T. S. Huang, "Visual interpretation of hand gestures for human-computer interaction: A review," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 19, no. 7, pp. 677-695, 1997.
- [2] S. Mitra and T. Acharya, "Gesture recognition: A survey," *IEEE Trans. Syst., Man, Cybern. C, Appl. Rev.*, vol. 37, no. 3, pp. 311-324, 2007.
- [3] C. Lugaresi et al., "MediaPipe: A Framework for Building Perception Pipelines," *arXiv preprint arXiv:1906.08172*, 2019.
- [4] F. Zhang, V. Bazarevsky, A. Vakunov, A. Tkachenka, G. Sung, C. L. Chang, and M. Grundmann, "MediaPipe Hands: On-device Real-time Hand Tracking," *arXiv preprint arXiv:2006.10214*, 2020.
- [5] C. Cortes and V. Vapnik, "Support-vector networks," *Machine learning*, vol. 20, no. 3, pp. 273-297, 1995.
- [6] J. Shotton et al., "Real-time human pose recognition in parts from single depth images," in *CVPR 2011*, pp. 1297-1304, 2011.
- [7] R. Wang, "Deep Learning for Sensor-based Activity Recognition," *Sensors*, 2019.
- [8] S. Oudah, A. Ali, and C. Edgar, "Hand Gesture Recognition Based on Computer Vision: A Review of Techniques," *Journal of Imaging*, vol. 6, no. 8, p. 73, 2020.
- [9] M. Kumar, "Depth-Sensor based Real-Time Hand Gesture Recognition," *IEEE Sensors Journal*, 2021.
- [10] L. Zhang et al., "Dynamic Time Warping for Trajectory Matching in HCI," *ACM CHI*, 2022.
- [11] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," *Commun. ACM*, vol. 60, no. 6, pp. 84-90, 2017.
- [12] N. Pugeault and R. Bowden, "Spelling it out: Real-time ASL finger-spelling recognition," in *IEEE International Conference on Computer Vision Workshops*, pp. 1114-1119, 2011.
- [13] P. Viola and M. Jones, "Rapid object detection using a boosted cascade of simple features," in *CVPR 2001*, vol. 1, pp. 1-511, 2001.
- [14] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural computation*, vol. 9, no. 8, pp. 1735-1780, 1997.
- [15] J. Brooke, "SUS: A 'Quick and Dirty' Usability Scale," in *Usability Evaluation in Industry*, London: Taylor and Francis, 1996.
- [16] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436-444, 2015.
- [17] D. Wu, L. Pigou, P. Kindermans, N. D. Schrauwen, J. Dambre, and P. Odobez, "Deep Dynamic Neural Networks for Multimodal Gesture Recognition," *Int. J. Comput. Vis.*, vol. 116, no. 3, pp. 317-328, 2016.
- [18] T. Starner and A. Pentland, "Real-time american sign language recognition using desk and wearable computer based video," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 20, no. 12, pp. 1371-1375, 1998.
- [19] H. Cooper, B. Holt, and R. Bowden, "Sign Language Recognition," in *Visual Analysis of Humans*, Springer, pp. 539-562, 2011.
- [20] M. O. Derawi, C. Nickel, P. Bours, and C. Busch, "Unobtrusive user-authentication on mobile phones using biometric gait recognition," in *IHH-MSP*, 2010.
- [21] R. E. Kalman, "A new approach to linear filtering and prediction problems," *Journal of basic Engineering*, 1960.
- [22] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.
- [23] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *JMLR*, vol. 12, pp. 2825-2830, 2011.
- [24] G. Bradski, "The OpenCV Library," *Dr. Dobbs's Journal of Software Tools*, 2000.
- [25] D. A. Norman, *The Design of Everyday Things*. Basic Books, 2013.